QuantRule — Architecture Description

QuantRule

2026-11-11

Contents

1 QuantRule — Architecture Description **1**
- 1.1 1. Context . . . 1
- 1.2 2. Containers . . . 1
- 1.3 3. Data flow . . . 2
- 1.4 4. Current code references . . . 2
- 1.5 5. Quality attributes . . . 2
- 1.6 6. Security and privacy . . . 3
- 1.7 7. Deployment . . . 3
- 1.8 8. Future work . . . 3
- 1.9 9. Evidence . . . 3

1 QuantRule — Architecture Description

- Status: Outline for the Tauri Platform; final version after Phase 3
- Claim type: ADR (Architecture Decision)

1.1 1. Context

QuantRule is a local-first research workstation for a solo systematic researcher. The user owns the candles and the machine; the product owns the deterministic pipeline and the local research memory. A static download page distributes Linux installers. A team cloud is future work.

1.2 2. Containers

Container	Responsibility	Current code
Desktop UI	Workspace, config editor, run view, analytics, memory	`apps/desktop` (Phase 3)
Application service	Use-case orchestration shared by UI and CLI	`crates/quantrule-app` (Phase 3)
Data engine	Normalization, ladder, ordering, gaps	`crates/quantrule-data`
Model engine	Readings, states, levels, events, progression, context	`crates/quantrule-model`
Backtest engine	Strategy, execution, portfolio, benchmark	`crates/quantrule-backtest`

Container	Responsibility	Current code
CLI shell	Headless command surface	`crates/quantrule-shell`
Local research memory	Workspaces, datasets, configs, runs, claims, notes	`crates/quantrule-app` + SQLite (Phase 3)
Download page	Static Linux installer distribution	`site/` (Phase 3)

1.3 3. Data flow

```
local candles
  -> data engine
  -> model engine
  -> strategy
  -> execution
  -> portfolio
  -> evidence + local research memory
  -> desktop UI / CLI
```

The engines are pure and depend one way. The application service owns I/O and orchestration. The UI and CLI do not compute indicators or rules.

1.4 4. Current code references

Concern	Path
Workspace and crate graph	`Cargo.toml`, `crates/`
Data engine	`crates/quantrule-data/src/lib.rs`
Model engine	`crates/quantrule-model/src/lib.rs`
Backtest engines	`crates/quantrule-backtest/src/lib.rs`
Shell and command surface	`crates/quantrule-shell/src/main.rs`, `docs/architecture/command-surface.md`
Determinism tests	`crates/quantrule-model/tests/`, `crates/quantrule-shell/tests/`
Architecture baseline	`docs/architecture/`
Current kata	`docs/architecture/kata.md`
Current characteristics	`docs/architecture/characteristics.md`

1.5 5. Quality attributes

Attribute	Design response
Reproducibility	Pure engines, fixed accumulation order, ordered collections, provenance
Temporal integrity	Availability bars, causality tests, as-of joins
Auditability	Artifact bundles, hashes, data version, config hash
Testability	Unit, golden, causality, determinism, shell, UI E2E
Deployability	Linux AppImage/deb; static download page

Attribute	Design response
Evolvability	Versioned run, config, and research-memory schemas
Cost	Local compute; no cloud runtime

1.6 6. Security and privacy

- No account, no cloud data store, no engine network calls.
- The only network path is the explicit `fetch` command and an optional update check.
- Strategies and candles stay local.
- The download page publishes installers and checksums only.

1.7 7. Deployment

- Desktop: Tauri Linux installers.
- Distribution: static page and GitHub Releases.
- Headless: existing CLI.
- No server, no database service, no cloud compute.

1.8 8. Future work

A collaborative team cloud is a future increment. It would add a control plane for accounts, config/run metadata, roles, and verification, but not candles or cloud backtesting. It requires a new ADR and characteristics review.

1.9 9. Evidence

Final evidence will be added after Phase 3: clean Linux install, UI E2E, byte-identical rerun, offline test, backup/restore, and the course checkpoints.